Aify

recurso gratuito

cómo hacemos los reels de aify sin cámara y sin editor

el proceso real de código e ia con el que montamos nuestros vídeos promocionales: guion, voz, sincronización palabra por palabra y una web que se graba sola. con el prompt para que lo repliques.

por dylan stillavato · fundador de aify · useaify.app

ANTES DE EMPEZAR

por qué hacemos esto así

los vídeos promocionales de aify no se graban ni se editan: **se renderizan**. son una página web con una animación en 3d que se convierte en vídeo fotograma a fotograma. sin cámara, sin timeline de premiere, sin freelance de edición.

01

por qué te lo cuento

porque es el mismo tipo de sistema que construimos en aify para nuestros clientes: procesos que antes eran manuales y ahora corren solos. este documento es la prueba, aplicada a algo muy concreto: hacer un anuncio.

02

no hace falta saber programar para entenderlo

vas a entender la lógica completa: qué se le pide a la ia, cómo se sincroniza todo al milisegundo y por qué se exporta fotograma a fotograma en vez de grabar la pantalla.

03

al final, el prompt para montarlo tú

un briefing técnico, listo para pegar en tu asistente de código (claude, cursor, lo que uses), para que te construya tu propia versión.

LA DIFERENCIA

esto no es otro editor de vídeo

un editor de vídeo trabaja sobre un timeline: arrastras clips, ajustas keyframes a ojo, exportas y cruzas los dedos con el timing. este sistema trabaja al revés: **todo se calcula** a partir de la voz, y el vídeo es solo la foto de ese cálculo, fotograma a fotograma.

EDITAR A MANO

- × ajustas cada subtítulo a ojo, escuchando
- × si cambias una frase, recolocas todo lo de después
- × grabar la pantalla en directo pierde fotogramas
- × cada efecto de sonido sale de un banco externo

RENDERIZAR CON CÓDIGO

- ✓ el karaoke se calcula solo, palabra por palabra
- ✓ cambias el guion y toda la línea de tiempo se recalcula
- ✓ se exporta fotograma a fotograma: cero tirones, timing exacto
- ✓ los efectos se generan con matemáticas, al milisegundo exacto

VISIÓN GENERAL

el proceso, de un vistazo

de un guion en una tabla a un mp4 terminado, en 5 pasos. cada uno alimenta al siguiente.

01 **el guion, escena por escena**
una tabla con lo que se dice en cada momento del vídeo — nada más.

02 **la voz, generada con ia**
una locución por frase, pidiéndole a la ia el ritmo y la energía en el propio prompt.

03 **medir y sincronizar con python**
se recorta el silencio de cada audio y se reparte el tiempo de cada frase palabra por palabra.

04 **la animación, como función del tiempo**
una sola página web donde todo (texto, 3d, efectos) se calcula a partir del segundo exacto del vídeo.

05 **exportar fotograma a fotograma**
un navegador real hace una captura por cada fotograma, y ffmpeg las une con el audio.

PASOS 1 Y 2

el guion y la voz con ia

el guion es literalmente una tabla: qué escena, qué se dice. sin más formato que ese.

escena	01 · el problema
locución	"le pides ideas a cualquier ia... y todas suenan igual."
escena	02 · presentamos
locución	"presentamos aify. la lluvia de ideas que sí conoce a tu cliente."

la voz

un audio por frase, con el estilo en el propio prompt

se genera cada línea por separado con un modelo de texto a voz (gemini tts), describiendo el ritmo dentro del mismo prompt: *"habla rápido, con energía y urgencia, en español de españa: [frase]"*. cada frase sale como un .wav suelto.

PASO 3 · LA PARTE QUE HACE QUE FUNCIONE

medir el audio y repartir el tiempo

esto es lo que hace que el subtítulo se ilumine palabra por palabra exactamente cuando se dice, sin tocarlo a mano.

recortar silencios

detectar dónde empieza y acaba la voz

se mide la energía del audio en ventanas de 20 ms (rms) y se recorta todo lo que hay antes y después de que empiece a hablar. así cada frase encaja justa en la línea de tiempo, sin huecos muertos.

repartir palabra por palabra

el truco del karaoke

cada frase se divide entre sus palabras de forma proporcional al número de letras (y un poco más si acaban en punto o coma, porque ahí se habla más despacio). el resultado: un timestamp exacto por palabra, sin escuchar el audio ni una vez a mano.

efectos de sonido, sin bancos de audio

generados con matemáticas

un "pop" es un tono que decae exponencialmente. un "whoosh" es un barrido de frecuencia. un "glitch" es ruido filtrado. un "boom" es una onda grave que cae. todo escrito en unas pocas líneas, colocado en el instante exacto del guion.

PASO 4 · LA IDEA CENTRAL DE TODO EL SISTEMA

el vídeo es una función del tiempo

no hay una animación "en marcha" que se pueda desincronizar. hay una única función, **render(t)**, que recibe el segundo exacto y coloca todo: qué texto se ve, dónde está cada elemento 3d, qué opacidad tiene cada capa. llamarla con el mismo t siempre pinta exactamente lo mismo.

IDEA SIMPLIFICADA

```
// se llama una vez por cada fotograma del vídeo final
function render(t) {
  // ¿qué frase toca a este segundo?
  const linea = buscarLinea(TIMING.lineas, t);

  // pinta cada palabra de un color u otro
  // según si ya se ha dicho o no
  subtitulo.html = linea.palabras
    .map(p => t >= p.momento ? resaltada(p) : apagada(p))
    .join(' ');

  // el logo 3d gira y se acerca según el momento
  camara.mirarA(0, 0, 0);
  nucleo.escala = interpolar(0.85, 1.2, progreso(t, ESCENA.presentamos));

  renderer.render(escena, camara);
}
```

PASO 5 · LA CLAVE PARA QUE NO SE NOTE QUE ES CÓDIGO

exportar fotograma a fotograma

en vez de reproducir la página y grabar la pantalla en tiempo real (con el riesgo de perder fotogramas), se hace lo contrario: se para el tiempo por completo.

01 **se abre la página con un navegador real**

controlado por código (playwright), no una captura de pantalla — así se aprovecha la gpu de verdad para el 3d.

02 **por cada fotograma del vídeo final**

se llama a `render(fotograma / fps)` y se hace una captura de ese instante exacto — el navegador no "avanza" solo, se le dice exactamente qué segundo pintar.

03 **ffmpeg une todas las capturas**

a la velocidad de fotogramas elegida, y le añade el audio (voz + efectos) generado en el paso 3.

por qué importa

timing perfecto siempre, sin importar la máquina

como cada fotograma se calcula por separado y no en tiempo real, da igual si el ordenador va lento ese día: el resultado es exactamente el mismo vídeo, fotograma por fotograma, cada vez que se exporta.

CÓPIALO Y PÉGALO

el prompt, para tu asistente de código

pégalo en claude, cursor o el asistente que uses. no es un prompt de texto: es un **briefing técnico** para que te construya el esqueleto del sistema completo.

```
Quiero construir un generador de vídeos promocionales que no se
graba en tiempo real: se renderiza fotograma a fotograma desde
una página web. Ayúdame a montar este sistema, paso a paso:

1. GUION
Un archivo simple con las frases que se van a decir, una por escena,
en orden.

2. VOZ CON IA
Genera un audio por frase con un modelo de texto a voz (Gemini TTS,
ElevenLabs...), describiendo el ritmo y el tono dentro del propio
prompt de la voz.

3. MEDIR Y SINCRONIZAR (Python)
- Recorta el silencio de cada audio detectando dónde empieza y acaba
  la energía real de la voz (RMS por ventanas de ~20 ms).
- Coloca cada frase en una línea de tiempo acumulada, con un hueco
  fijo entre frases y pausas manuales donde haga falta dramatismo.
- Reparte el tiempo de cada frase palabra por palabra, proporcional
  al número de letras (algo más en las que acaban en punto o coma),
  para poder resaltar cada palabra justo cuando se dice (karaoke).
- Genera los efectos de sonido con matemáticas puras, sin bancos de
  audio: un tono que decae para un "pop", ruido filtrado para un
  "glitch", un barrido de frecuencia para un "whoosh".
- Mezcla voz + efectos en un único audio normalizado, y escribe un
  archivo de datos (json) con: duración total, línea de tiempo por
  frase y por palabra, e instante de cada efecto.

4. LA ANIMACIÓN, COMO FUNCIÓN DEL TIEMPO
Una sola página HTML con una función render(t) que recibe el segundo
exacto del vídeo y coloca todo en base a t: qué texto se ve, en qué
posición está cada elemento, la opacidad de cada capa. Nada de
keyframes a mano: todo se calcula a partir de t y del json del
paso 3. Usa three.js si quieres profundidad 3D, o solo CSS/canvas
si prefieres 2D.

5. EXPORTAR FOTOGRAMA A FOTOGRAMA
Abre la página con un navegador real controlado por código
(Playwright), y por cada fotograma del vídeo final:
- llama a render(fotograma / fps)
- haz una captura de ese instante exacto
Une todas las capturas con ffmpeg a la velocidad de fotogramas
```

SI LO TUYO ES EL CONTENIDO, NO EL CÓDIGO

en aify hacemos lo mismo con tu contenido

este documento es un ejemplo de cómo pensamos: automatizar lo que antes era manual. en aify aplicamos esa misma lógica a las ideas, guiones y calendarios de tus clientes — sin que tengas que montar tú el pipeline.

incluye

ideas + guiones + copies + calendario, con ia

con el contexto real de cada cliente guardado una sola vez: su tono, su sector, su audiencia.

pruébalo 30 días gratis

pide tarjeta, pero no se cobra nada si cancelas antes de que termine la prueba.

useaify.app